

COURSE

Основы программирования

JS

Обработка событий

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Передача параметров в обработчик события **4**

Встроенные обработчики **5**

Свойства обработчиков событий **6**

Слушатели событий **6**

Объект Event **8**

Распространение событий **9**

Восходящие события 9

Нисходящие события 11

События мыши **12**

События клавиатуры **14**

Литература и другие источники *17*

Вопросы для самопроверки *17*

ПЛАН ЛЕКЦИИ

1. Передача параметров и обработчики событий;
2. Встроенные обработчики;
3. Свойства обработчиков событий;
4. Слушатели событий;
5. Объект Event;
6. Распространение событий;
7. События мыши;
8. События клавиатуры;

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Для взаимодействия с пользователем в JavaScript определен механизм событий. Например, когда пользователь нажимает кнопку, то возникает событие нажатия кнопки. В коде JavaScript мы можем определить возникновение события и как-то его обработать.

В JavaScript есть следующие типы событий:

- События мыши (перемещение курсора, нажатие мыши и т.д.).
- События клавиатуры (нажатие или отпускание клавиши клавиатуры).
- События жизненного цикла элементов (например, событие загрузки веб-станицы).
- События элементов форм (нажатие кнопки на форме, выбор элемента в выпадающем списке и т.д.).
- События, возникающие при изменении элементов DOM.
- События, возникающие при касании на сенсорных экранах.
- События, возникающие при возникновении ошибок.

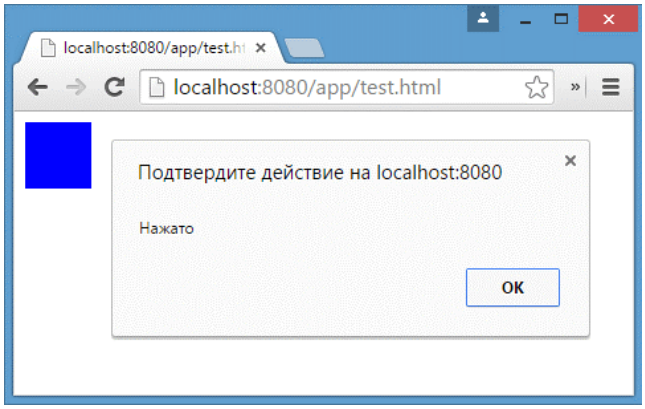
Рассмотрим простейшую обработку событий. Например, на веб-странице у нас есть следующий элемент div:

```
<div id="rect" onclick="alert('Нажато')" style="width:50px; height:50px; background-color: blue;"></div>
```

Здесь определен обычный блок div, который имеет атрибут onclick, который задает обработчик события нажатия на блок div. То есть, чтобы обработать какое-либо событие, нам надо определить для него обработчик. Обработчик представляет собой код на языке JavaScript. В данном случае обработчик выглядит довольно просто:

```
alert('Нажато')
```

И при нажатии на кнопку будет выскакивать сообщение:



Также можно было бы вынести все действия по обработке события в отдельную функцию:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
</head>
<body>
<div id="rect" onclick="displayMessage()" style="width:50px; height:50px; background-color:blue;"></div>
<script>
function displayMessage(){
    alert('Нажато');
}
</script>
</body>
</html>
```

Теперь обработчиком события будет выступать функция displayMessage.

Передача параметров в обработчик события

В обработчик можно передавать параметры. Например, мы можем передать текущий объект, на котором возникает событие:

```
<a href="page1.html" onclick="return handler(this)">Станица 1</a>
<script>
function handler(obj){

    alert(obj.href);
    return false;
}
</script>
```

Ключевое слово this указывает на текущий объект ссылки, на которую производится нажатие. И в коде обработчика мы можем получить этот объект и обратиться к его свойствам, например, к свойству href.

Кроме того, надо отметить, что здесь обработчик возвращает результат. Хотя в первом примере с блоком div от обработчика не требовалось возвращения результата. Дело в том, что для некоторых обработчиков можно подтвердить или остановить обработку события. Например, нажатие на ссылку должно привести к переадресации. Но возвращая из обработчика false, мы можем остановить стандартный путь обработки события, и переадресации не будет. Если же возвращать значение true, то событие обрабатывается в стандартном порядке.

Если же мы вовсе уберем возвращении результата, то событие будет обрабатываться, как будто возвращается значение true:

```
<a href="page1.html" onclick="handler(this)">Станица 1</a>
<script>
function handler(obj){

    alert(obj.href);
```

```
}
</script>
```

Кроме непосредственно элемента-источника события в обработчик мы можем передавать объект event. Этот объект не определяется разработчиком, это просто аргумент функции обработчика, который хранит всю информацию о событии. Например:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #rect{
        width:50px;
        height:50px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="rect" onclick="handler(event)"></div>
<script>
function handler(e){

    alert(e.type); // получаем тип события
}
</script>
</body>
</html>
```

В данном случае с помощью свойства type объекта event получаем тип события (в данном случае тип click).

Встроенные обработчики

В прошлой теме были рассмотрены ***встроенные обработчики*** (inline event handler), которые определяются в коде элемента с помощью атрибутов:

```
<div id="rect" onclick="handler(event)"></div>
```

Хотя этот подход прекрасно работает, но он имеет кучу недостатков:

- Код html смешивается с кодом JavaScript, в связи с чем становится труднее разрабатывать, отлаживать и поддерживать приложение.
- Обработчики событий можно задать только для уже созданных на веб-странице элементов. Динамически создаваемые элементы в этом случае лишаются возможности обработки событий.
- К элементу для одного события может быть прикреплен только один обработчик.
- Нельзя удалить обработчик без изменения кода.

Свойства обработчиков событий

Проблемы, которые возникают при использовании встроенных обработчиков, были призваны решить ***свойства обработчиков***. Подобно тому, как у html-элементов есть атрибуты для обработчиков, так и в коде javascript у элементов DOM мы можем получить свойства обработчиков, которые соответствуют атрибутам:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #rect{
        width:50px;
        height:50px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="rect"></div>
<script>
function handler(e){

    alert(e.type);
}
document.getElementById("rect").onclick = handler;
</script>
</body>
</html>
```

В итоге нам достаточно взять свойство onclick и присвоить ему функцию, используемую в качестве обработчика. За счет этого код html отделяется от кода javascript.

Стоит также отметить, что в обработчик события браузер автоматически передает объект Event, хранящий всю информацию о событии. Поэтому также мы можем получить этот объект в функции обработчика в качестве параметра.

Слушатели событий

Несмотря на то, что свойства обработчиков решают ряд проблем, которые связаны с использованием атрибутов, в то же время - это также не оптимальный подход. Еще один способ установки обработчиков событий представляет использование слушателей.

Для работы со слушателями событий в JavaScript есть объект EventTarget, который определяет методы addEventListener() (для добавления слушателя) и removeEventListener() для удаления слушателя. И поскольку html-элементы DOM тоже являются объектами EventTarget, то они также имеют эти методы. Фактически слушатели представляют те же функции обработчиков.

Метод addEventListener() принимает два параметра: название события без префикса on и функцию обработчика этого события. Например:

```
<!DOCTYPE html>
<html>
```

```
<head>
    <meta charset="utf-8" />
    <style>
    #rect{
        width:50px;
        height:50px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="rect"></div>
<script>
var rect = document.getElementById("rect");

rect.addEventListener("click", function (e) {
    alert(e.type);
});
</script>
</body>
</html>
```

То есть в данном случае опять же обрабатывается событие click. И также можно было бы в качестве второго параметра название функции:

```
function handler(e){

    alert(e.type);
}
var rect = document.getElementById("rect");

rect.addEventListener("click", handler);
```

Удаление слушателя аналогично добавлению:

```
rect.removeEventListener("click", handler);
```

Преимуществом использования слушателей является и то, что мы можем установить для одного события несколько функций:

```
var clicks = 0;
function handlerOne(e){

    alert(e.type);
}
function handlerTwo(e){

    clicks++;
    var newNode = document.createElement("p");
    newNode.textContent = "произошло нажатие " + clicks;
    document.body.appendChild(newNode);
}
var rect = document.getElementById("rect");
```

```
// прикрепляем первый обработчик
rect.addEventListener("click", handlerOne);
// прикрепляем второй обработчик
rect.addEventListener("click", handlerTwo);
```

Объект Event

При обработке события браузер автоматически передает в функцию обработчика в качестве параметра объект Event, который инкапсулирует всю информацию о событии. И с помощью его свойств мы можем получить эту информацию:

- **bubbles**: возвращает true, если событие является восходящим. Например, если событие возникло на вложенном элементе, то оно может быть обработано на родительском элементе.
- **cancelable**: возвращает true, если можно отменить стандартную обработку события.
- **currentTarget**: определяет элемент, к которому прикреплен обработчик события.
- **defaultPrevented**: возвращает true, если был вызван у объекта Event метод preventDefault().
- **eventPhase**: определяет стадию обработки события.
- **target**: указывает на элемент, на котором было вызвано событие.
- **timeStamp**: хранит время возникновения события.
- **type**: указывает на имя события.

Например:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #rect{
        width:50px;
        height:50px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="rect"></div>
<script>
function handler(event){
    console.log("Тип события: " + event.type);
    console.log(event.target);
}
var rect = document.getElementById("rect");
rect.addEventListener("click", handler);
</script>
</body>
</html>
```

Причем в данном случае свойство target представляет собой элемент, поэтому мы можем манипулировать им, как и любыми другими узлами и элементами DOM. Например, изменим фоновый цвет:

```
function handler(e){
```

```
    e.target.style.backgroundColor = "red";
}
```

С помощью ***метода preventDefault()*** объекта Event мы можем остановить дальнейшее выполнение события. В ряде случаев этот метод не играет большой роли. Однако в некоторых ситуаций он может быть полезен. Например, при нажатии на ссылку мы можем с помощью дополнительной обработки определить, надо ли переходить по ссылке или надо запретить переход. Или другой пример: пользователь отправляет данные формы, но в ходе обработки в обработчике события мы определили, что поля формы заполнены неправильно, и в этом случае мы также можем запретить отправку.

Например, запретим переход по ссылке после 12 часов:

```
<a href="http://google.com" id="link">Поиск</a>
<script>
function linkHandler(e){
    var date = new Date();
    var hour = date.getHours();
    console.log(hour);
    if(hour>12){
        e.preventDefault();
        document.write("После 12 переход запрещен");
    }
}
var link = document.getElementById("link");
link.addEventListener("click", linkHandler);
</script>
```

Распространение событий

Когда мы нажимаем на какой-либо элемент на станице и генерируется событие нажатия, то это событие может распространяться от элемента к элементу. Например, если мы нажимаем на блок div, то также мы нажимаем и на элемент body, в котором блок div находится. То есть происходит распространение события.

Есть несколько форм распространения событий:

- Восходящие: событие распространяется вверх по дереву DOM от дочерних узлов к родительским.
- Нисходящие: событие распространяется вниз по дереву DOM от родительских узлов к дочерним, пока не достигнет того элемента, на котором это событие возникло.

Восходящие события

Рассмотрим восходящие (bubbling) события, которые распространяются в верх по дереву DOM. Допустим, у нас есть следующая веб-страница:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #blueRect{
        width:100px;
```

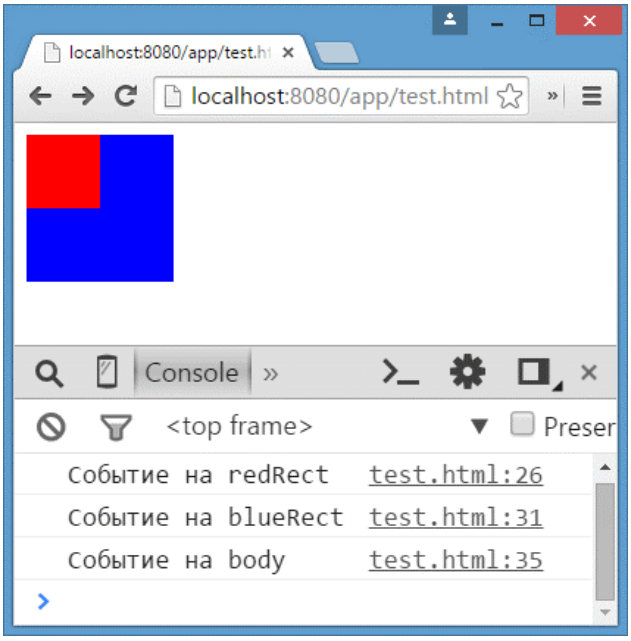
```
            height:100px;
            background-color:blue;
        }
        #redRect{
            width:50px;
            height:50px;
            background-color:red;
        }
        </style>
    </head>
    <body>
    <div id="blueRect">
        <div id="redRect"></div>
    </div>

    <script>
    var redRect = document.getElementById("redRect");
    redRect.addEventListener("click", function(){
        console.log("Событие на redRect");
    });

    var blueRect = document.getElementById("blueRect");
    blueRect.addEventListener("click", function(){
        console.log("Событие на blueRect");
    });

    document.body.addEventListener("click", function(){
        console.log("Событие на body");
    });
    </script>
    </body>
    </html>
```

Если мы нажмем на вложенный div, то событие пойдет к родительскому элементу div и далее к элементу body:



Надо сказать, что подобное поведение не всегда является желательным. И в этом случае мы можем остановить распространение событие с помощью метода stopPropagation() объекта Event:

```
var redRect = document.getElementById("redRect");
redRect.addEventListener("click", function(e){
    console.log("Событие на redRect");
    e.stopPropagation();
});
```

И в результате нажатия событие будет обработано только обработчиком для redRect.

Нисходящие события

События также могут быть нисходящими (capturing). Для их использования в метод addEventListener() в качестве третьего необязательного параметра передается логическое значение true или false, которое указывает, будет ли событие нисходящим. По умолчанию все события восходящие.

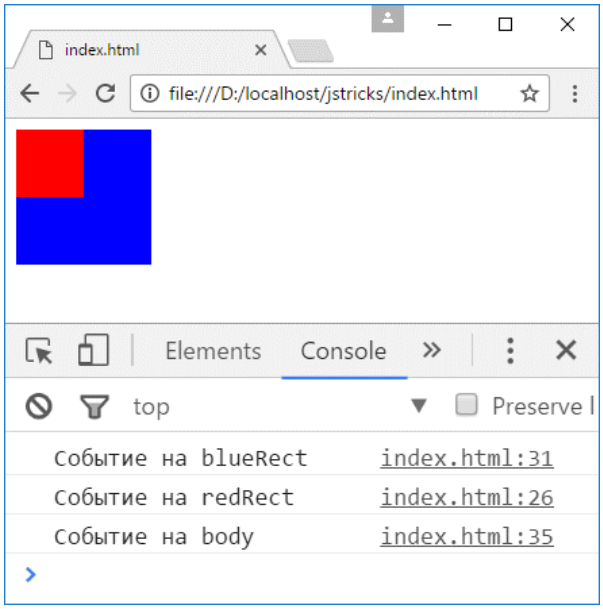
Возьмем ту же веб-станицу, только изменим ее код javascript:

```
var redRect = document.getElementById("redRect");
redRect.addEventListener("click", function(){
    console.log("Событие на redRect");
}, true);

var blueRect = document.getElementById("blueRect");
blueRect.addEventListener("click", function(){
    console.log("Событие на blueRect");
}, true);

document.body.addEventListener("click", function(){
    console.log("Событие на body");
}, true);
```

Теперь события будут распространяться в обратном порядке:



События мыши

Одну из наиболее часто используемых событий составляют события мыши:

- **click**: возникает при нажатии указателем мыши на элемент.
- **mousedown**: возникает при нахождении указателя мыши на элементе, когда кнопка мыши находится в нажатом состоянии.
- **mouseup**: возникает при нахождении указателя мыши на элементе во время отпускания кнопки мыши.
- **mouseover**: возникает при вхождении указателя мыши в границы элемента.
- **mousemove**: возникает при прохождении указателя мыши над элементом.
- **mouseout**: возникает, когда указатель мыши выходит за пределы элемента.

Например, обработаем события mouseover и mouseout:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #blueRect{
        width:100px;
        height:100px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="blueRect"></div>

<script>
function setColor(e){
```

```
    if(e.type==="mouseover")
        e.target.style.backgroundColor = "red";
    else if(e.type==="mouseout")
        e.target.style.backgroundColor = "blue";
}
var blueRect = document.getElementById("blueRect");
blueRect.addEventListener("mouseover", setColor);
blueRect.addEventListener("mouseout", setColor);
</script>
</body>
</html>
```

Теперь при наведении указателя мыши на блок blueRect он будет окрашиваться в красный цвет, а при уходе указателя мыши - блок будет обратно окрашиваться в синий цвет.

Объект Event является общим для всех событий. Однако для разных типов событий существуют также свои объекты событий, которые добавляют ряд своих свойств. Так, для работы с событиями указателя мыши определен объект MouseEvent, который добавляет следующие свойства:

- **altKey**: возвращает true, если была нажата клавиша Alt во время генерации события.
- **button**: указывает, какая кнопка мыши была нажата.
- **clientX**: определяет координату X окна браузера, на которой находился указатель мыши во время генерации события.
- **clientY**: определяет координату Y окна браузера, на которой находился указатель мыши во время генерации события.
- **ctrlKey**: возвращает true, если была нажата клавиша Ctrl во время генерации события.
- **metaKey**: возвращает true, если была нажата во время генерации события метаклавиша клавиатуры.
- **relatedTarget**: определяет вторичный источник возникновения события.
- **screenX**: определяет координату X относительно верхнего левого угла экрана монитора, на которой находился указатель мыши во время генерации события.
- **screenY**: определяет координату Y относительно верхнего левого угла экрана монитора, на которой находился указатель мыши во время генерации события.
- **shiftKey**: возвращает true, если была нажата клавиша Shift во время генерации события.

Определим координаты клика:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    #blueRect{
        width:100px;
        height:100px;
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="blueRect"></div>
```

```
<script>
function handleClick(e){
    console.log("screenX: " + e.screenX);
    console.log("screenY: " + e.screenY);
    console.log("clientX: " + e.clientX);
    console.log("clientY: " + e.clientY);
}
var blueRect = document.getElementById("blueRect");
blueRect.addEventListener("click", handleClick);
</script>
</body>
</html>
```

События клавиатуры

Другим распространенным типом событий являются события клавиатуры.

- **keydown**: возникает при нажатии клавиши клавиатуры и длится, пока нажата клавиша
- **keyup**: возникает при отпускании клавиши клавиатуры
- **keypress**: возникает при нажатии клавиши клавиатуры, но после события keydown и до события **keyup**. Надо учитывать, что данное событие генерируется только для тех клавиш, которые формируют вывод в виде символов, например, при печати символов. Нажатия на остальные клавиши, например, на Alt, не учитываются.

Для работы с событиями клавиатуры определен объект KeyboardEvent, который добавляет к свойствам объекта Event ряд специфичных для клавиатуры свойств:

- **altKey**: возвращает true, если была нажата клавиша Alt во время генерации события.
- **charCode**: возвращает символ Unicode для нажатой клавиши (используется для события keypress).
- **keyCode**: возвращает числовое представление нажатой клавиши клавиатуры.
- **ctrlKey**: возвращает true, если была нажата клавиша Ctrl во время генерации события.
- **metaKey**: возвращает true, если была нажата во время генерации события метаклавиша клавиатуры.
- **shiftKey**: возвращает true, если была нажата клавиша Shift во время генерации события.

Например, мы можем с помощью клавиш клавиатуры перемещать элемент на веб-странице:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <style>
    html, body{
        margin:0;
        overflow:hidden;
    }
    #blueRect{
        width:100px;
        height:100px;
```

```
        background-color:blue;
    }
    </style>
</head>
<body>
<div id="blueRect"></div>

<script>
function moveRect(e){
    var blueRect = document.getElementById("blueRect");
    // получаем стиль для blueRect
    var cs = window.getComputedStyle(blueRect);

    var left = parseInt(cs.marginLeft);
    var top = parseInt(cs.marginTop);

    switch(e.keyCode){
        case 37:  // если нажата клавиша влево
            if(left>0)
                blueRect.style.marginLeft = left - 10 + "px";
            break;
        case 38:   // если нажата клавиша вверх
            if(top>0)
                blueRect.style.marginTop = top - 10 + "px";
            break;
        case 39:   // если нажата клавиша вправо
            if(left < document.documentElement.clientWidth - 100)
                blueRect.style.marginLeft = left + 10 + "px";
            break;
        case 40:   // если нажата клавиша вниз
            if(top < document.documentElement.clientHeight - 100)
                blueRect.style.marginTop = top + 10 + "px";
            break;
    }
}

addEventListener("keydown", moveRect);
</script>
</body>
</html>
```

В данном случае обрабатывается событие keydown. В обработчике moveRect с помощью метода window.getComputedStyle() получаем стиль элемента blueRect. А затем из этого стиля выбираем значения свойств marginLeft и marginTop.

С помощью свойства e.keyCode получаем нажатую клавишу. Список кодов клавиш клавиатуры можно посмотреть на сайте http://www.javascriptkeycode.com/.

Здесь нам интересуют четыре клавиши: вверх, вниз, влево, вправо. Если одна из них нажата, производим действия: увеличение или уменьшение отступа элемента от верхней или левой границы. Ну и чтобы элемент не выходил за границы окна, проверяем предельные значения с помощью

document.documentElement.clientWidth (ширина корневого элемента) и document.documentElement.clientHeight (высота корневого элемента).

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

1. Дакетт Джон. HTML и CSS. Разработка и дизайн веб-сайтов. Пер. с англ. – Эксмо, 2017. – 480 с.
2. Прасти Н. Введение в ECMAScript 6. Изд. ДМК, 2016. – 176 с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Как для элементов формы передавать параметры в обработчик событий?
2. Какие недостатки проявляются при использовании встроенных обработчиков событий?
3. Для чего предназначены слушатели событий и как они работают?
4. В чем заключается отличие входящих от исходящих событий при их распространении?
5. Перечислите все события мыши, с которыми можно работать в JavaScript?
6. Перечислите все события клавиатуры, с которыми можно работать в JavaScript?